Aim:

(A) Implement linear regression using Python by calculating the slope (a) and intercept (b) manually through formulas and plot the best fit line for a given dataset using Matplotlib.

(B) Build a simple linear regression model that predicts output values based on a single input feature using Python and machine learning libraries.

Objective: To generate a dataset with random noise using NumPy

- To apply linear regression formulas to calculate the slope and intercept.
- To apply linear regression for solving real-world prediction problems.
- To understand how to fit a model using training data and make predictions on unseen data.

Technologies Used

- Anaconda Distribution: Open-source Python/R distribution for scientific computing and data science.

- Jupyter Notebook: A web-based IDE for writing and executing Python code.

- Python: Programming language used for ML model development.

Step-by-Step Implementation

A Part:

```
import numpy as np
import pandas as pd
x = np.linspace(1,6,10)
x
```

```
array([1.        , 1.55555556, 2.11111111, 2.66666667, 3.22222222,
       3.77777778, 4.33333333, 4.88888889, 5.44444444, 6.        ])
```

```
temp=[]
for n in x:
    t=(np.random.choice(range(1,3)))
    temp.append(n+t)
    y=np.array(temp)
```

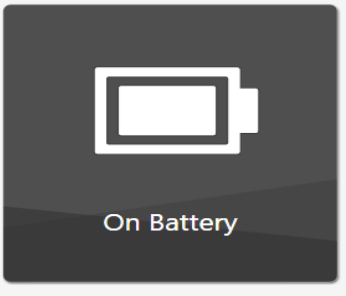
```
y
```

```
array([3.        , 3.55555556, 4.11111111, 4.66666667, 4.22222222,
       4.77777778, 6.33333333, 6.88888889, 6.44444444, 7.        ])
```

```
m, c = np.polyfit(x, y, 1)
best_fit_y = m * x + c

# Plot
plt.figure(figsize=(8, 6))
plt.scatter(x, y, label='Data Points')
plt.plot(x, best_fit_y, color='red', linewidth=2, label=f'Best Fit Line: y = {m:.2f}x + {c:.2f}')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Scatter Plot with Best Fit Line")
plt.legend()
plt.grid(True)
plt.show()
```

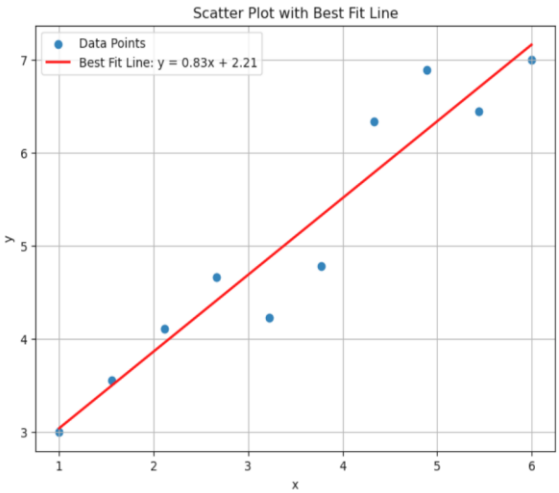
B Part:

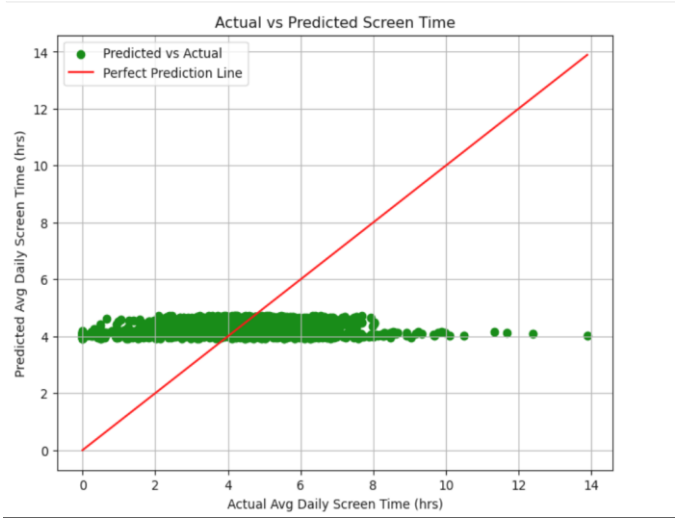
```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Step 2: Load the dataset
df = pd.read_csv("Indian_Kids_Screen_Time.csv")

# Step 3: Select features and target
features = ['Age', 'Educational_to_Recreational_Ratio']
target = 'Avg_Daily_Screen_Time_hr'

X = df[features]
y = df[target]

# Step 4: Handle missing values
X = X.dropna()
y = y.loc[X.index]

# Step 5: Train/Test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```




```
# Step 6: Train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 7: Predictions
y_pred = model.predict(X_test)

# Step 8: Visualization - Actual vs Predicted with dashed line
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='green', label='Predicted vs Actual')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='-', label='Perfect Prediction Line
plt.xlabel("Actual Avg Daily Screen Time (hrs)")
plt.ylabel("Predicted Avg Daily Screen Time (hrs)")
plt.title("Actual vs Predicted Screen Time")
plt.legend()
plt.grid(True)
plt.show()

# Step 9: Evaluation
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)
```

Output:






```
Mean Squared Error (MSE): 2.9285431076401034
Root Mean Squared Error (RMSE): 1.7112986611460033
R² Score: 0.01261882998392072
```

Learning Outcomes:

- Able to implement a basic machine learning model using Python
- Understand the working of linear regression with one independent and one dependent variable.
- Learn how to split data and fit a model using scikit-learn.